BEGINNER EXPLAINER

ChoiceForge

How a GPU kernel can accelerate a real travel demand modeling component - from the first idea through complete ActivitySim results.

Written for readers with no background in transportation modeling, probability, or GPU computing.

8.114x	1.388x	1.064x
utility-pipeline microbenchmark	complete trip-destination component	complete 34-step 50k-household model

Phase 15 update: compact generated CUDA is exact and 1.097x faster for the qualified destination component, but the 50,000-household scale candidate is not promoted.

Who this guide is for

This guide is for a curious high school student. You do not need to know transportation planning, probability, programming, or computer hardware. By the end, you should understand:

- what a travel demand model tries to predict;
- how a computer turns possible choices into probabilities;
- why the same calculation must be repeated millions of times;
- what CPUs, GPUs, and GPU kernels do;
- what ChoiceForge changes;
- what the early benchmark results do and do not prove; and
- why faster modeling could matter to communities.

The one-minute version

A city may want to know what could happen if it adds a bus line, changes a toll, builds housing, or closes a bridge. It cannot test every idea in the real world first, so planners use a **travel demand model**: a computer simulation of how people may decide where, when, why, and how to travel.

Modern models can represent millions of people and many possible choices. For each person, the computer scores the available choices, turns the scores into probabilities, and uses a controlled random number to select one outcome. It repeats this work again and again for trips, tours, destinations, travel modes, and times of day.

A normal processor, called a **CPU**, is very flexible but has a modest number of powerful processing cores. A **GPU** has many more, simpler cores and is good at doing the same kind of arithmetic for a huge number of records at once. A **GPU kernel** is a small program designed to run across those GPU cores.

ChoiceForge is an open-source experiment that moves a particularly repetitive travel-choice calculation to an NVIDIA GPU. Its most important idea is **fusion**: calculate utility scores, apply availability rules, compute a logsum, and make the choice in one GPU operation, without first storing an enormous table of intermediate scores.

The first synthetic test compared the GPU with a readable NumPy reference. Phase 1 added a much stronger 24-core CPU competitor. Phase 2 captured 4,477 real mandatory-tour scheduling decisions from ActivitySim, and every GPU choice matched. It also found a problem: transferring a 151.6-megabyte table made the GPU slower. Phase 3 replaced that table with 22.0 megabytes of compact ingredients and made the captured calculation 3.83 times faster than a purpose-built 48-thread CPU version. Phases 4 and 5 installed and reused the GPU path in real scheduling. Phase 6 handled real destination work. Phase 7 batched repeated setup and added nested logit on the GPU. Phase 8 moved the work to pinned current ActivitySim and a public 50,000-household run. Phase 9 then used the much larger public MTC geography. It confirmed an exact GPU destination result and also caught a scheduling mismatch before that path could be claimed as correct. Phase 11 repeated the full-geography destination experiment three times and established the current production result: the complete 50,000-household model was 1.064 times faster with byte-for-byte identical outputs. Phase 12 built the foundation for moving the large utility equations themselves onto the GPU. Phase 13 completed the strict CPU answer key. Phase 14 generated GPU code from that same recipe and matched the CPU exactly on 30 real public-model batches. Phase 15 connected it to the real model, removed the giant intermediate table, proved a destination-component win, and found an honest scale limit.

1. What is travel demand modeling?

Imagine a region with homes, schools, offices, stores, roads, sidewalks, rail lines, and buses. Transportation planners ask questions such as:

- How many people might travel during the morning rush?
- Where might they go?
- Will they drive, walk, bike, take transit, or share a ride?
- Which roads or transit services might become crowded?
- Who benefits from a project, and who may be left out?
- How might results change if fuel prices, fares, jobs, or housing change?

A travel demand model does not read minds or predict one person's future with certainty. It creates a plausible regional picture by applying observed patterns and mathematical rules to many simulated people.

People, households, and zones

Models usually begin with a **synthetic population**. These are made-up records that resemble the region's real population without being a list of actual named residents. A record may describe a household's size, income, vehicles, workers, and students.

The region is divided into geographic areas called **zones**. A zone may contain homes, jobs, schools, shops, or transit stations. The model also needs information about travel between zones, such as distance, driving time, transit time, cost, and whether a route is available. These travel measurements are often called **skims**.

Activities, tours, and trips

People travel because they want to do activities. Going from home to school is a trip. Going from home to school, then to a store, then home is often treated as a **tour**: a chain of trips that begins and ends at the same anchor, usually home or work.

An activity-based model may answer a sequence of linked questions:

```
Who is traveling?
    -> What activity are they doing?
    -> Where will they go?
    -> What time will they travel?
    -> Which travel mode will they use?
    -> Which trips and tours result?
```

These decisions affect one another. A faraway destination may make walking unrealistic. A late return time may make transit unavailable. A household with one vehicle and two workers may face competition for the car.

2. How does a model represent a choice?

Suppose Maya must choose among walking, taking the bus, and riding in a car. The model gives every option a **utility score**. Utility is not electricity here. It is a numerical score for how attractive an option is according to the model.

A simplified utility equation looks like this:

```
utility = constant + (time x time weight) + (cost x cost weight) + other terms
```

A negative time weight means longer trips are less attractive. A negative cost weight means expensive trips are less attractive. The weights are estimated from travel surveys or other observations. Different people can receive different scores because income, age, vehicle access, location, and other facts may matter.

Availability comes first

Not every option is possible. Walking may be unavailable for a very long trip. A transit route may not exist. A person may not have access to a car. The model applies an **availability rule** so an impossible option receives no chance of being chosen.

From scores to probabilities

Raw utility scores are not probabilities. A common model uses this rule:

```
probability of option i = exp(utility i) / sum of exp(utility for every available option)
```

`exp` is a mathematical function that turns higher scores into much larger positive numbers. Dividing by the total makes all probabilities add to 1, or 100 percent.

Here is a small example:

Option	Utility score	exp(score)	Probability
Walk	0	1.000	9.0%
Bus	1	2.718	24.5%
Car	2	7.389	66.5%

The car is most likely, but it is not guaranteed. This matters because real people with similar situations do not all make the same decision.

Turning probabilities into one choice

The model takes a random draw between 0 and 1. It lays the probabilities end to end:

```
Walk: 0.000 to 0.090
Bus:  0.090 to 0.335
Car:  0.335 to 1.000
```

If the draw is 0.600, the model chooses car. If it is 0.200, it chooses bus. This is called **inverse cumulative distribution sampling**, but the important idea is simple: larger probability ranges are more likely to catch the draw.

ChoiceForge does not invent random numbers inside the GPU. The calling model supplies them. That design lets the CPU and GPU use the exact same draws, which makes fair comparison possible and preserves ActivitySim's rules for repeatable simulations.

What is a logsum?

A **logsum** summarizes how good the full set of choices is:

```
logsum = log(sum of exp(utility for every available option))
```

In the example, the logsum is about 2.407. If a fast, affordable new bus service improves one option, the logsum rises. Models often pass this accessibility-like value into later decisions.

Computers must calculate logsums carefully. Very large utility scores can make `exp(score)` overflow, like a calculator displaying an error for a number that is too large. The standard safe method subtracts the largest utility before exponentiating and adds it back afterward. This is called **stable logsum-exp**. ChoiceForge uses this method.

3. Why does this become a computing problem?

The three-option example is tiny. A regional model may have:

- millions of people, households, tours, or trips;
- dozens of alternatives for mode or time choices;
- hundreds or thousands of possible destinations;
- many variables in every utility equation; and

- dozens of model steps that repeat similar calculations.

If one million choosers each consider 32 alternatives, the model evaluates 32 million chooser-option combinations in just one component. Destination choice can be much larger.

A straightforward program may create a rectangular **utility matrix** with one row per chooser and one column per alternative. For one million choosers and 32 alternatives stored as 32-bit numbers, that table alone uses 128 megabytes. The program may then create more tables for exponentials, probabilities, or cumulative probabilities. Moving and storing these intermediate values can take as much time as the arithmetic.

4. CPU, GPU, and kernel in everyday language

A **CPU** is the general-purpose brain of a computer. Think of it as a small team of highly trained chefs. Each chef can handle complicated instructions, switch tasks quickly, and make decisions.

A **GPU** was designed to calculate many screen pixels at once. Think of it as a very large kitchen crew. Each worker is less independent, but thousands of workers can perform the same short recipe on different ingredients at the same time.

A **GPU kernel** is that short recipe. It says what one group of GPU threads should calculate. Launching a kernel sends many copies of the recipe across the data.

The GPU is not automatically faster. Three costs matter:

1. **Transfer:** data may need to cross from ordinary computer memory to GPU memory and back.
2. **Launch overhead:** starting a GPU kernel takes a small amount of time.
3. **Parallel work:** the job must be large and regular enough to keep many GPU cores busy.

Small or irregular jobs may be faster on the CPU. A good GPU design must move enough connected work together and avoid unnecessary transfers.

5. The main ChoiceForge idea: fuse the pipeline

A less efficient pipeline might do this:

```
CPU creates inputs
    -> GPU computes and stores every utility
    -> another operation reads utilities and computes probabilities
    -> another operation reads probabilities and selects choices
    -> results return to CPU
```

ChoiceForge's fused linear kernel aims to do this:

```
chooser features + alternative weights + constants + availability + random draw
    -> one fused GPU kernel
    -> selected alternative + logsum
```

Inside the kernel, one GPU block handles one chooser. Threads in the block work on the alternatives together. They calculate utility scores, ignore unavailable options, find the maximum score, form safe exponential weights, sum them, select the option containing the supplied random draw, and return only the useful outputs.

The intermediate chooser-by-alternative utility matrix is never written to large global GPU memory. Temporary values stay close to the GPU cores in faster **shared memory**. This reduces memory traffic and storage.

The milestone-one kernel supports:

- a fixed set of 1 to 1,024 alternatives;
- linear utility equations using 32-bit floating-point numbers;

- availability rules;
- stable logsums;
- externally supplied random draws; and
- exact comparison of selected alternatives with a CPU reference.

It does not yet support all of ActivitySim's expression language, estimation mode, or destination sets with thousands of alternatives. Phase 7 supports the canonical MTC 21-mode nested-logit tree, but not every irregular ActivitySim choice model.

6. How the prototype is kept honest

Speed is useless if the model silently changes its answers. The project therefore starts with a readable NumPy CPU implementation called the **reference** or **oracle**. The GPU result is checked against it.

The validation rules are:

- give the CPU and GPU identical inputs and identical random draws;
- preserve the same alternative order and boundary rules;
- require every selected alternative to match exactly;
- measure the numerical difference between CPU and GPU logsums;
- test unavailable alternatives and invalid rows; and
- report mismatches instead of hiding them.

Small floating-point differences are normal because parallel GPU operations may add numbers in a different order. It is like adding a long list of rounded decimals from left to right versus pairing them first: the last digit can differ. The selected choices still must match, and logsum differences must stay within an explicit tolerance.

The Python 3.11 ActivitySim and CUDA integration environment now passes 75 tests. These include exact comparison with ActivitySim's real Numba choice function, compact expression compilation, segmented destination batches, categorical flags, nested-logit validation, current-version fallback forwarding, real scheduling integration, GPU tests using 33 and 190 alternatives, a safe expression interpreter, skim-table adapters, shadow checks, the strict CPU reference, and the generated strict CUDA evaluator.

7. What was benchmarked?

A **benchmark** is a controlled timing experiment. ChoiceForge's first benchmark uses synthetic, meaning computer-generated, data. Each test has:

- 32 alternatives;
- 16 features per chooser;
- 10,000, 100,000, or 1,000,000 choosers; and
- 32-bit floating-point calculations.

It compares three timings:

1. **NumPy materialized:** the readable CPU reference creates the utility matrix.
2. **GPU including transfers:** inputs move to the GPU, the kernel runs, and outputs move back.
3. **GPU resident:** needed arrays are assumed to already be in GPU memory.

The GPU-resident number shows the potential of a future pipeline that keeps related model steps on the GPU. The transfer-inclusive number is the more conservative measure for a single isolated call.

All numbers below are medians after warm-up on an NVIDIA RTX A4000 with 16 GB of memory.

Choosers	NumPy CPU	GPU incl. transfers	GPU resident	End-to-end speedup	Resident speedup
10,000	5.156 ms	0.845 ms	0.361 ms	6.10x	14.26x
100,000	50.614 ms	3.587 ms	1.037 ms	14.11x	48.79x
1,000,000	499.375 ms	36.610 ms	8.118 ms	13.64x	61.51x

For all three sizes:

- choice mismatches: **0**;
- maximum logsum error: less than **0.000001**; and
- the fused kernel avoided allocating the global utility matrix.

At one million choosers, this original comparison was about 13.64 times faster. But NumPy was built as a readable correctness reference, not the strongest possible CPU competitor. Phase 1 therefore added a fused Numba implementation that spreads choosers over all 24 physical CPU cores.

Phase 1: comparison with the strongest CPU baseline

For the original 32-alternative, 16-feature shape, the 24-core fused CPU was much stronger:

Choosers	Best 24-core CPU	GPU incl. transfers	Transfer-inclusive result	GPU resident
10,000	0.390 ms	1.260 ms	GPU is 0.31x as fast	0.810 ms
100,000	5.560 ms	4.170 ms	GPU is 1.33x faster	1.190 ms
1,000,000	59.260 ms	34.520 ms	GPU is 1.72x faster	7.110 ms

This result is more realistic and less dramatic. The GPU loses on the smallest job because transfer and launch costs dominate. Its advantage grows with the amount of work.

Phase 1 also tested a shape based on mandatory tour scheduling in prototype MTC. It has 190 time alternatives and 69 feature rows:

Choosers	Best 24-core CPU	GPU incl. transfers	Speedup	GPU resident	Choice mismatches
10,000	10.760 ms	2.560 ms	4.19x	0.970 ms	0
100,000	109.660 ms	20.380 ms	5.38x	6.730 ms	2

The wider calculation gives each transferred row much more arithmetic, which suits the GPU. The two differences at 100,000 rows occurred when random draws were less than three ten-millionths from a probability boundary. NumPy chose one option while both fused Numba and CUDA chose the immediately following option. This tiny but important result means the project needs a clear 64-bit or mixed-precision rule before claiming exact ActivitySim equivalence.

The wider test also discovered and fixed a real GPU synchronization bug. The original 32 alternatives fit inside one group of 32 GPU threads, called a warp. With 190 alternatives, several warps shared temporary memory. One warp could reuse that memory before another had finished reading it. A new synchronization barrier fixes the race, and tests above 32 alternatives prevent it from returning.

Phase 2: replaying real ActivitySim scheduling data

Phase 1 copied the *shape* of tour scheduling, but its people and feature values were generated by a random-number program. Phase 2 uses real model records from ActivitySim's public `prototype_mtc` example.

ActivitySim's **mandatory tour scheduling** step chooses when work and school tours begin and end. For example, one option might leave at 8 a.m. and return at 5 p.m. The utility equation considers 52 to 59 active terms, including:

- the traveler's type, income, and work or school situation;
- the proposed start time, end time, and duration;
- whether another tour already occupies part of the day;
- a mode-choice logsum describing the quality of available travel modes; and
- constants that make some departure, arrival, and duration ranges more or less attractive.

The capture program observes ActivitySim at a documented internal boundary. It records the evaluated expression terms, feasible alternatives, coefficients, probabilities, random draws, and final selected positions. It does not change ActivitySim's rules or generate new draws.

The public sample produced six mandatory scheduling batches: first and second tours for work, school, and university students. Together they contain 4,477 tour choices. The largest batch contains 3,381 first work tours and 642,390 feasible chooser-alternative rows.

Did the answers stay the same?

Yes. All 4,477 GPU choices matched ActivitySim exactly. A separate 64-bit replay of ActivitySim's probability tables also had zero mismatches. The largest batch's maximum utility difference was less than 0.00000044. In the full model output, every tour's selected time alternative, start time, and end time matched the earlier warm Sharrow run.

Was the GPU faster?

The answer depends on where the input data begins:

Copies of the real batch	Choosers	CPU	GPU including transfers	GPU already resident	Resident speedup
1	3,381	8.241 ms	15.545 ms	0.961 ms	8.57x
2	6,762	20.346 ms	32.298 ms	1.326 ms	15.34x
4	13,524	43.035 ms	62.427 ms	2.242 ms	19.20x

For the larger rows, Phase 2 repeats captured real tours and their original draws. It does not invent synthetic travelers. Repeating records is a throughput test, not a claim that the public model contains more households.

The resident kernel clearly wins. The transfer-inclusive path clearly loses. The reason is that the current lowerer expands the expressions into a large table of term values: 151.6 megabytes for the native largest batch. Sending that table across the computer's PCIe connection takes much longer than the GPU arithmetic.

An analogy is a very fast bakery on the other side of town. Baking takes one minute, but delivering every separate ingredient takes fifteen minutes. Buying a faster oven will not solve the delivery problem. The next design must send compact ingredients once, prepare more of them inside the GPU, and keep them there for connected model steps.

What has Phase 2 proved?

It has proved that the ragged scheduling kernel can process the real ActivitySim alternative sets with exact choices, and that its resident computation is faster than a strong CPU boundary on this machine. It has also disproved the idea that simply adding a GPU call around an expanded term table makes the isolated component faster.

At that Phase 3 boundary, it had not yet proved that the entire mandatory scheduling component or full ActivitySim model ran faster. Phases 4 through 6 later added those wider proofs. Stateful timetable expressions and other ActivitySim front-end work still run on the CPU.

Phase 3: sending compact ingredients instead of finished terms

Phase 2 sent one value for every expression and every feasible alternative. Many values repeated the same traveler fact or the same time-alternative fact thousands of times. It was like shipping 59 finished ingredient bowls for every possible schedule.

Phase 3 sends a compact package instead:

- traveler facts are stored once per tour chooser;
- start, end, and duration are stored once for each of 190 time alternatives;
- each feasible row carries a small alternative ID;
- only the mode-choice logsum and seven timetable-dependent values stay row-specific; and
- ActivitySim still supplies the exact random draw.

The compact package for the largest real batch is 22.046 megabytes instead of 151.604 megabytes, an 85.46 percent reduction.

What does the compiler do?

A **compiler** translates instructions from one form into another. Here it reads the real ActivitySim expressions, checks that every operation and column is supported, and writes matching CUDA arithmetic into the kernel. It supports the scheduling model's numbers, arithmetic, comparisons, and Boolean combinations such as "A and B." Unknown names and unsupported function calls cause a clear error instead of a silent wrong answer.

The same expressions also generate a strong CPU version compiled by Numba and spread across 48 CPU threads. This makes the comparison much harder and fairer than comparing the GPU only with a simple Python loop.

Copies of the real batch	Choosers	Compact input	48-thread CPU	GPU including transfers	GPU resident	Inclusive speedup
1	3,381	22.046 MB	9.481 ms	2.478 ms	0.181 ms	3.83x
2	6,762	44.091 MB	18.443 ms	4.681 ms	0.271 ms	3.94x
4	13,524	88.179 MB	37.132 ms	9.348 ms	0.410 ms	3.97x
8	27,048	176.355 MB	69.735 ms	17.787 ms	0.709 ms	3.92x

All six mandatory-scheduling batches still have zero choice mismatches. The largest utility difference from the Phase 2 calculation is 0.0000229, and the largest GPU-versus-CPU logsum difference is 0.00000293.

Phase 3 proves transfer-inclusive superiority at this captured kernel boundary. It still does not time the work needed to build the compact tables inside ActivitySim, calculate mode-choice logsums, update the timetable, or map results back into pandas tables. Those steps belong in the next complete-component test.

Phase 4: putting the GPU inside the real ActivitySim component

Phase 4 performs that complete-component test. ActivitySim now has an explicit setting that chooses either its normal scheduling calculation or ChoiceForge. It is not a hidden test hook. ActivitySim still decides which alternatives are possible, calculates mode-choice logsums, supplies the controlled random numbers, updates each person's timetable, and writes the output tables.

The first integrated version was correct but slower. A profile showed that the GPU work was not the problem. Repeatedly asking seven timetable questions for 642,390 rows took about 4.03 seconds. A day in this model has only 21 time slots, so ChoiceForge now answers those questions once on a small traveler-by-time-slot grid and looks up the required answers. That reduced the largest timetable stage to about 0.331 seconds.

Two three-trial experiments measured the result:

Test	Cached Sharrow median	ChoiceForge median	Speedup	Schedule mismatches
Normal full-model runs	5.515 s	4.925 s	1.120x	0
Same saved checkpoint before scheduling	8.599 s	8.014 s	1.073x	0

The timer includes the work that Phase 3 left outside: mode-choice logsums, timetable calculations, compact packing, ActivitySim's random-number call, transfers, GPU work, result mapping, and timetable updates. It even includes fresh-process GPU setup. All three normal ChoiceForge runs produced exactly the same 9,806 final `tdd`, start, end, duration, destination-logsum, and mode-choice-logsum values as the cached-Sharrow reference.

Why is the complete-component speedup 1.120x instead of 3.83x? The 3.83x number describes the calculation ChoiceForge directly replaces. The complete component also performs substantial mode-choice logsum and table work that remains the same in both versions. Speeding up one part of a larger job gives a smaller improvement to the whole job. This is an example of **Amdahl's law**: total speedup is limited by the work that was not accelerated.

Phase 5: one backend for the scheduling family

Mandatory tours are only one kind of tour. A **non-mandatory tour** might be shopping, eating out, escorting someone, or recreation. A **joint tour** is shared by members of a household. An **at-work subtour** starts and ends at a workplace, such as going out for lunch. These components use related choice math, but they do not have identical columns or timetable rules.

Phase 5 taught the compiler to understand those differences without creating three separate GPU systems. It can turn text categories such as "escort" into compact yes-or-no columns, recognize both "equals" and "does not equal," and discover whether a timetable row belongs to a person or a tour. Calculations that are safe and repetitive go into the fused GPU kernel. Special state-changing timetable work remains explicit and checked.

Three complete-model trials measured all four ActivitySim scheduling timers:

Scheduling component	Cached Sharrow median	ChoiceForge median	Speedup
Mandatory	5.515 s	5.077 s	1.086x
Joint	0.671 s	0.504 s	1.331x
Non-mandatory	1.560 s	0.550 s	2.836x
At-work subtour	0.309 s	0.202 s	1.530x
All four together	**8.045 s**	**6.325 s**	**1.272x**

The four-component scheduling family uses 21.4 percent less time, saving a median 1.720 seconds. This was not caused by one lucky run: the slowest ChoiceForge trial was still faster than the fastest cached-Sharrow trial. Non-mandatory scheduling improves most because it contains a large amount of repeated choice work but does not carry the same expensive mode-choice-logsum work as mandatory scheduling.

Correctness is checked more strongly than before. For every Phase 5 trial, the final accessibility, households, joint-tour participants, land use, persons, tours, and trips files have exactly the same SHA-256 fingerprints as the cached-Sharrow reference. A SHA-256 fingerprint is a long number made from every byte in a file; changing even one character almost certainly changes the fingerprint. This covers 9,806 tour rows and 23,583 trip rows, not just a few selected columns.

There was an important Phase 5 limit. Its separately run sessions did not prove an all-model gain because unrelated run-to-run variation was larger than the scheduling savings. Phase 6 later closed that evidence gap with an alternating A/B experiment.

8. A negative result that teaches an important lesson

The project also tested only ActivitySim's final probability-sampling operation for 100,000 choosers and 32 alternatives.

Method	Time	Result compared with warm ActivitySim Numba
ActivitySim Numba on CPU	2.515 ms	baseline
GPU including transfers	2.591 ms	0.97x, slightly slower
GPU resident	0.212 ms	11.86x faster

There were zero choice mismatches. However, after paying to transfer the probability table, the isolated GPU call was slightly slower than ActivitySim's warm CPU function.

This is useful evidence, not a failure to hide. It shows that sending only the final tiny step to the GPU is a poor design. The strong performance idea depends on fusing utility evaluation, logsum calculation, and sampling, then keeping data on the GPU across connected steps.

9. How ActivitySim fits in

ActivitySim is a popular open-source activity-based travel modeling framework written in Python. ChoiceForge is designed as a possible specialized computing backend for parts of ActivitySim, not as a new travel demand model that replaces its behavioral rules.

The project integrated ChoiceForge with ActivitySim 1.4 and ran the public 25-zone prototype MTC example from start to finish:

- 5,000 households;
- 8,212 persons;
- 34 completed model steps;
- 70.568 seconds total runtime; and
- 580.8 MB peak unique memory.

That early 70.568-second run was only a framework baseline. Phase 4 later ran the full model with ChoiceForge handling mandatory scheduling, and Phase 5 expanded it to all four tour-scheduling components.

ActivitySim can also use **Sharrow**, an optimized expression system. Its compile run eventually completed in 27.3 minutes and used about 15 GB of unique memory. That compilation is not a fair production timing. After the cache existed, three warm runs took 61.650, 59.119, and 61.790 seconds, for a median of 61.650 seconds. Their final household, person, tour, and trip files were identical.

The warm profile showed where time is spent. Trip destination used 25.3% of total time. Mandatory tour scheduling used 9.1%, and trip scheduling used 8.4%. Mandatory scheduling is a practical first integration target, but it cannot reduce total runtime by 10% on its own because it is only 9.1% of the model. A reusable scheduling backend should target several scheduling components together.

Phase 6: making destination work large enough

Trip destination asks where an intermediate stop happens. For every candidate stop, the model also estimates how attractive the travel from the trip origin to the stop would be and how attractive the remaining travel to the main tour destination would be. Those are the two **directions** called OD and DP.

The first GPU replay exposed 30 small groups, separated by trip number and purpose. Sending each group to the GPU separately was a bad bargain: the GPU took about 15.032 milliseconds including transfers, while the CPU needed only 2.646 milliseconds. Think of sending 30 nearly empty delivery trucks instead of loading one truck.

ChoiceForge then packed all 30 groups into one segmented kernel launch. A segment label tells the kernel which purpose's coefficients to use, while offsets tell it where each traveler's uneven list of candidate places begins and ends. The packed job contained 3,971 choosers and 53,927 candidate rows. It took 0.847 milliseconds on the GPU including transfers versus 1.241 milliseconds for a strengthened, batched Numba CPU version: **1.464 times faster**. All 3,971 chosen positions matched, and logsums differed by at most about nine millionths.

The crossover test explains when to use it. At 28,570 rows the GPU was slower. At 39,706 rows it was faster. On this computer, this shape should be packed to roughly 35,000 rows before GPU dispatch. That threshold must be remeasured on other computers.

The first proven whole-model improvement

The live ActivitySim loop still asks for one small purpose group at a time, so turning on the new GPU sampler there would make it slower. Phase 6 instead removed duplicated work from the much larger directional-logsum calculation. It preserved ActivitySim's OD random draws, then its DP draws, but processed the shared deterministic work together.

Six fresh full-model runs alternated the old and new paths in the order A1/B1/A2/B2/A3/B3. Trip destination fell from a 16.307-second median to 14.238 seconds, **1.145 times faster**. All 34 model steps fell from 59.209 to 56.264 seconds, **1.052 times faster**. Every new-path component run beat every old-path component run. Seven final result files were byte-for-byte identical in every trial.

This distinction matters: the 1.464x number proves the packed CUDA kernel boundary; the 1.145x number proves the complete destination component; and the 1.052x number proves this complete example model. They are not interchangeable.

Phase 7: stop repeating setup, then accelerate the actual nest

Phase 7 asked what was still consuming the 14.238 seconds. ActivitySim repeated a 70-step preparation recipe once for each of ten trip purposes, even though most of that recipe was the same. ChoiceForge now puts the purposes for one trip number into one larger worksheet. The recipe runs three times - once for trip number 1, 2, and 3 - instead of 30 times. Sampling and final decisions remain separate, so each traveler keeps the same random draws and behavioral rules.

The next boundary is called **nested logit**. Imagine grouping 21 travel modes into folders: driving modes, walking/biking, transit, and ride-hailing. Some folders contain smaller folders. Nested logit combines the scores inside each folder and then combines the folders. It represents that two similar choices, such as two transit services, are more closely related than transit and driving.

We captured every real 21-mode score table used by destination choice: 30 batches and 107,854 rows. ActivitySim's dataframe reducer needed a median of 486.626 milliseconds for the complete captured sequence. The fused GPU kernel, including sending 18.119 MB to the GPU and returning the answers, needed 120.737 milliseconds in the longer 31-trial test: **4.030 times faster**. Even its slowest recorded GPU trial was faster than the fastest CPU trial. Its largest logsum difference was about 0.0000000000000036.

The full-model test alternated the Phase 6 and Phase 7 programs six times. Trip destination fell from 12.179 to 10.281 seconds, **1.185 times faster**. All 34 model steps fell from 54.459 to 52.166 seconds, **1.044 times faster**. Every Phase 7 run beat every Phase 6 run, and all seven final result files were byte-for-byte identical in all six trials.

Why is a 40x kernel only a 1.044x whole-model improvement? The kernel replaces less than half a second of CPU reduction. Most remaining time evaluates 404 behavioral expressions, samples destinations, runs other model components, and organizes data. This is Amdahl's law: speeding up a small slice cannot speed up the whole pie by the same amount.

Phase 8: current software and ten times more households

Earlier complete-model tests used 5,000 households. Phase 8 asked two harder questions. Does the integration still work with a pinned current ActivitySim development version? Does the advantage remain when the public workflow grows to 50,000 households across 190 zones?

The answer on this workstation is yes. The experiment alternated three regular ActivitySim runs with three ChoiceForge runs, always starting a fresh process:

Measured boundary	Regular ActivitySim median	ChoiceForge median	Speedup
Four scheduling components	30.1 s	23.7 s	**1.270x**
Trip destination	30.9 s	21.8 s	**1.417x**
All 34 model steps	147.225 s	131.812 s	**1.117x**

ChoiceForge saved a median of 15.413 seconds for the whole run. Every one of the three ChoiceForge runs was faster than every regular run. The seven final files matched byte for byte in all six trials. Those files describe 50,000 households, 111,130 people, 142,761 tours, and 350,751 trips.

The scale also shows what happens inside the computer. The largest scheduling call contains 9,561,750 feasible schedule rows for 50,325 choosers. Its compact input is 327.920 megabytes. The GPU part takes about 0.4 seconds, but preparing timetable facts, packing the data, retrieving controlled random numbers, and mapping the answer make the entire boundary about 3.3 seconds. This means the next scheduling improvement should focus on preparation and data reuse, not only on making the GPU arithmetic faster.

Phase 8 also captured a much larger nested-logit calculation: 30 real batches, 3,186,130 rows, and 535.270 megabytes of 64-bit mode scores. In 11 alternating trials, ActivitySim's CPU reducer needed a 4.646627-second median. The GPU, including both transfers, needed 0.125627 seconds: **36.988 times faster**. Every GPU trial beat every CPU trial, and the largest logsum difference was about 0.0000000000000053.

That 36.988x result is for one captured mathematical boundary. The 1.117x number is the honest complete-model result. The rest of ActivitySim still has to create people and tours, evaluate expressions, sample places, organize tables, and write outputs.

Phase 9: much larger geography and a useful stop sign

Phase 9 downloaded the public full-geography Prototype MTC data. The full data has 2,875,192 households, 7,566,527 people, and 1,454 zones. That is 7.7 times as many zones as Phase 8. A zone is a geographic area such as a neighborhood or traffic-analysis area, so more zones make the travel calculations much larger and more varied.

The whole population needs far more memory than this workstation has. The published configuration describes a 64-process, 432-gigabyte-RAM setup. The local test therefore used 50,000 households but kept all 1,454 zones. It ran one fresh regular ActivitySim process and one fresh ChoiceForge process.

For this test, ChoiceForge accelerated only trip destination. It left the schedule choices on the regular ActivitySim path. The final accessibility, household, participant, land-use, person, tour, and trip files were byte for byte identical. They describe 50,000 households, 132,536 people, 175,579 tours, and 442,682 trips.

Measured boundary	Regular ActivitySim	ChoiceForge	Observed result
Trip destination	39.2 s	28.0 s	**1.400x faster**
All 34 model steps	198.794 s	187.010 s	**1.063x faster**

This is one correctly matched pair, not yet a final statistical claim. A high-memory computer must run several alternating pairs before those numbers can be called reliable medians.

Phase 9 also found nine schedule choices that changed at a floating-point boundary when the experimental scheduling GPU path was enabled at 100,000 households. One later decision changed because the earlier difference affected the model's sequence of controlled random numbers. ChoiceForge therefore turns that scheduling path off for this configuration. This is not a failure to hide: it is exactly why the project checks complete output files rather than trusting a fast-looking timer. A faster answer that changes the model is not acceptable.

10. What the current evidence proves

The project can currently support these narrow statements:

- The fused linear-choice GPU kernel runs correctly on the tested NVIDIA hardware and software setup.
- It produced the same choices as the CPU reference in the reported synthetic benchmarks.
- Its logsums were numerically extremely close to the reference.
- For the reported large synthetic workload, it was much faster than the readable, materializing NumPy reference, even when transfers were included.
- For the scheduling-shaped 10,000-row workload, it was 4.19 times faster than a fused 24-core Numba CPU including transfers, with zero choice mismatches.
- The GPU advantage depends strongly on problem shape and size: it lost to the fused CPU on the 10,000-row, 32-alternative workload.
- Avoiding a full utility matrix can reduce intermediate memory use.
- ActivitySim-compatible random draws and final sampling rules can be preserved.
- Offloading sampling alone is not worthwhile when transfer time is included.
- Warm Sharrow production runtime and component shares are now measured reproducibly.
- Phase 2 replayed 4,477 real mandatory-tour choices with zero GPU mismatches.
- The real scheduling kernel is 8.57 times faster when inputs are GPU-resident, while the transfer-inclusive path is slower at 0.53 times CPU speed.
- Phase 3 reduces that input by 85.46 percent and makes the native transfer-inclusive GPU boundary 3.83 times faster than a generated 48-thread CPU implementation.
- Phase 4 makes the complete mandatory scheduling component 1.120 times faster than cached Sharrow in normal runs and 1.073 times faster in matched-checkpoint runs.
- Phase 5 makes the complete four-component scheduling family 1.272 times faster, led by a 2.836 times non-mandatory scheduling speedup.
- Seven substantive final CSV files are byte-identical across all Phase 5 trials, including all 9,806 tours and 23,583 trips.
- Phase 6 makes the packed real destination kernel 1.464 times faster than a batched Numba CPU including transfers, with zero mismatches across 3,971 choices.
- Phase 6 makes the complete trip-destination component 1.145 times faster and the complete 34-step prototype model 1.052 times faster in interleaved trials.
- Seven substantive final CSV files are byte-identical across all six Phase 6 A/B trials.
- Phase 7's transfer-inclusive nested-logit reducer is 4.030 times faster than ActivitySim's pandas reducer at the captured aggregate boundary, with maximum absolute error of 3.6e-15.
- Phase 7 makes trip destination 1.185 times faster and the complete example model 1.044 times faster than Phase 6; every optimized run beats every baseline run.

- Seven substantive final CSV files are byte-identical across all six Phase 7 A/B trials.
- Phase 8's 3,186,130-row nested-logit replay is 36.988 times faster on the GPU including transfers, with maximum absolute error of 5.4e-15.
- On pinned current ActivitySim at 50,000 households, the four scheduling components are 1.270 times faster, trip destination is 1.417 times faster, and all 34 models are 1.117 times faster.
- All three Phase 8 optimized runs beat all three baselines, and seven substantive CSVs are byte-identical across all six runs, including all 142,761 tours and 350,751 trips.
- On public full MTC geography with 1,454 zones, Phase 9's destination-only GPU configuration has byte-identical outputs in one 50,000-household A/B pair and an observed 1.400x trip-destination ratio. It is a scale gate, not a median claim.
- The Phase 9 scheduling experiment found nine changed choices at 100,000 households, so that GPU path is disabled for full geography until a precision safeguard resolves them.

11. What the current evidence does not prove

It does **not** yet prove that:

- every ActivitySim model will run faster;
- every complete model will be faster;
- a complete model will be 13.64 times faster;
- arbitrary ActivitySim utility expressions can run in the kernel;
- destination choice with thousands of irregular alternatives is solved;
- results will be identical on every GPU and software version; or
- a faster calculation makes the behavioral model more accurate.

The NumPy reference remains useful for correctness, and Phase 1 includes a strong fused 24-core CPU competitor. Phase 2 supplies the real scheduling replay, Phase 3 supplies transfer-inclusive scheduling-kernel superiority, Phase 4 supplies the first complete-component comparison, Phase 5 supplies breadth, Phase 6 supplies a destination replay, Phase 7 supplies batched setup plus a live nested-logit GPU boundary, Phase 8 supplies pinned-current 50,000-household evidence, and Phase 9 supplies a full-geography scale gate. A broader claim still requires a differently structured public model, other GPUs, high-memory repetition, and independent reproduction.

12. From promising prototype to convincing result

A responsible roadmap looks like this:

Step 1: Expand the supported math - completed for four scheduling components

Phase 3 compiles the real scheduling arithmetic and Boolean expressions directly from compact traveler and alternative columns. Phase 5 adds categorical assignments, inequalities, and different timetable owners. Stateful timetable functions remain precomputed primitives, and other ActivitySim components may require more operations.

Step 2: Build a configured ActivitySim backend - completed for the tour-scheduling family

Phase 4 adds an explicit setting, preserves ActivitySim-owned random draws and timetable updates, and falls back to ActivitySim for unsupported tracing or estimation paths. Phase 5 reuses that contract for four scheduling components.

Step 3: Handle large destination choices - completed for the prototype model

Phase 6 packs 30 real ragged destination segments into one launch and proves a transfer-inclusive GPU win. Phase 7 batches live preprocessing across purposes and adds the 21-mode nested-logit GPU reducer. Much larger regional alternative sets remain future work.

Step 4: Prove correctness on public benchmark data - completed for the prototype scheduling family

Phase 2 checks all six prototype MTC mandatory-scheduling batches and records every numerical tolerance. Phase 5 verifies byte-identical final files after four scheduling components. Phase 6 checks all 30 destination batches and hashes all six A/B model trials. Phase 8 repeats the proof at 50,000 households on current ActivitySim. Phase 9 repeats the destination test across 1,454 public MTC zones and disables the scheduling path when its exactness gate fails. A differently structured public model is still needed before making a general claim.

Step 5: Run fair performance comparisons - completed for four scheduling components

Phase 5 compares three cached-Sharrow and three transfer-inclusive ChoiceForge scheduling runs. Phase 6 strengthens the design with alternating A/B processes and proves both component and whole-model gains. Other models and hardware still need the same treatment.

Step 6: Publish a reproducible evidence package

Include exact commands, pinned dependencies, raw timing samples, correctness reports, benchmark data instructions, and results from more than one GPU. Let other researchers reproduce or challenge the claim.

13. Why faster travel modeling could matter

Faster computation does not automatically make a better transportation decision. But it can change what analysts have time to study.

More scenarios

Instead of testing only a few projects, an agency could examine more combinations of fares, services, land use, road pricing, and policy assumptions.

Better uncertainty analysis

No forecast is certain. Faster models can support many runs with different population growth, fuel prices, remote-work rates, or random seeds. The range of results may be more informative than one forecast.

Faster feedback

Modelers could find data or configuration problems sooner, shorten development cycles, and respond more quickly to community questions.

Lower intermediate memory pressure

Fused calculations that avoid giant temporary tables may allow larger problems or reduce memory bottlenecks.

Important cautions

Speed cannot repair biased survey data, missing travel options, unrealistic assumptions, or unfair project goals. GPUs use electricity and require specialized hardware and skills. Public decisions still need transparency, equity analysis, local knowledge, and human judgment.

14. Frequently asked questions

Does the GPU predict travel differently?

It should implement the same model rules. The goal is to calculate them faster, not invent a different behavior model.

Does zero choice mismatch mean every number is identical?

Not always. Some kernel-level logsums differ by tiny amounts because floating-point arithmetic can be ordered differently in parallel, even when the selected alternative is unchanged. In Phases 5, 6, and 8, however, all seven substantive final CSV files were byte-for-byte identical in the reported A/B trials.

Why use 32-bit numbers?

They use half the memory of 64-bit numbers and are usually faster on GPUs. Whether they are accurate enough must be tested for each model. ActivitySim-compatible final sampling also has a 64-bit path in the adapter.

Why not move the entire model to the GPU immediately?

ActivitySim contains varied expressions, tables, joins, tracing, estimation tools, and irregular choice sets. A small, well-tested component creates a safer foundation than a large rewrite whose errors are hard to locate.

Is a GPU always faster?

No. Small workloads, frequent transfers, branching logic, or underused cores can make a GPU slower. The isolated sampling benchmark demonstrated this directly.

Which speedup is the headline?

Use the number for the boundary being discussed. Phase 3's scheduling kernel is 3.83x, Phase 7's smaller nested-logit reducer is 4.030x, and Phase 8's much larger nested-logit replay is 36.988x. At the 50,000-household scale, Phase 8 makes the four complete scheduling components 1.270x faster, complete trip destination 1.417x faster, and the complete model 1.117x faster. Phase 9's 1.400x destination ratio uses much larger 1,454-zone geography, but is only one matched pair. Only a carefully repeated full-model number should be called a whole-model speedup.

15. Mini glossary

Term	Plain-English meaning
ActivitySim	An open-source framework for activity-based travel demand models.
Amdahl's law	The idea that total speedup is limited by parts of a job that were not accelerated.
Alternative	One possible choice, such as walk, bus, or car.
Availability	A rule saying whether an alternative is possible.
Benchmark	A controlled experiment that measures speed and correctness.
Chooser	The person, household, tour, or trip making a simulated decision.
Compiler	A program that translates checked instructions into another executable form.
Compact representation	Data stored without unnecessarily repeating the same facts.
CPU	A flexible general-purpose processor with a modest number of powerful cores.
CUDA	NVIDIA's platform for programming its GPUs.
Feature	An input fact, such as travel time, cost, income, or vehicle access.
Floating point	The computer's approximate way of storing non-whole numbers.
Fusion	Combining connected calculations so intermediate data is not repeatedly stored or moved.
GPU	A processor with many simpler cores suited to large parallel calculations.
GPU kernel	A small program executed by many GPU threads.
Logsum	A summary of the attractiveness of all available alternatives.
Nested logit	A choice calculation that groups related alternatives before combining them.
Probability	A number from 0 to 1 describing how likely an outcome is.
Shared memory	Small, fast GPU memory close to threads in one block.
Skim	A table of travel time, distance, cost, or related values between places.
Synthetic population	Anonymous simulated households and people resembling a real region.
Utility	A model score representing how attractive an alternative is.

16. The bottom line

ChoiceForge asks a focused question: can a common bundle of travel-choice calculations be redesigned as one correctness-first GPU kernel that avoids huge intermediate tables?

The answer is now more precise. The fused kernel can decisively beat a strong 24-core CPU when each chooser has enough alternatives and features, but it loses on small, narrow jobs. Phase 1 also found a multi-warp synchronization defect and two near-boundary choice differences, demonstrating why wider correctness tests and precision rules matter as much as speed.

Phases 3 through 9 advanced integration through full MTC geography. Phase 8 improved whole-model time with exact outputs. Phase 9 confirmed exact destination results across a far larger zone system and held scheduling back until precision improves. Phase 11 then repeated that full-geography destination test and made the outcome stronger. Phase 12 identified the utility-calculation opportunity. Phase 13 completed the exact CPU answer key. Phase 14 generated the

matching GPU evaluator and proved exact agreement on real public-model batches, while keeping it safely in shadow mode.

17. The latest evidence: replicated destination speedup and a safer path to GPU utilities

This section updates the earlier phases. It uses two labels deliberately:

- **Proven production result** means a complete public ActivitySim run finished, its final files matched exactly, and the timing was repeated.
- **Foundation or microbenchmark** means a smaller, controlled engineering test succeeded. It is useful evidence, but it is not yet a claim that a whole ActivitySim model got faster.

Phase 10: a guardrail, not a shortcut

The full-geography scheduling experiment revealed a subtle risk. A GPU and CPU can both follow the same-looking formula yet round a last digit differently. If a random draw lands exactly near the boundary between two choices, that last digit can change the selected alternative. One changed early choice can also change the controlled sequence of later random draws.

Phase 10 added a **shadow guard**. Think of it as a second answer key running beside an experimental route. The established CPU/Sharrow result remains authoritative. The GPU is allowed to report success only if its shadow comparison passes; otherwise the system deliberately returns the CPU result. This is slower than blindly using the GPU, but it prevents a silent model change while the arithmetic issue is investigated.

Phase 11: the current headline result

Phase 11 focused on the part that was already exact: trip destination. It used the public Prototype MTC full geography with 1,454 zones and 50,000 households. Three fresh, interleaved A/B pairs were run: regular ActivitySim, then ChoiceForge, with the order alternated to reduce the chance that unrelated computer activity picked the winner.

Measured boundary	Regular ActivitySim median	ChoiceForge median	Result
Trip destination	39.7 s	28.6 s	**1.388x faster**
All 34 model steps	202.492 s	190.380 s	**1.064x faster**

The complete run saved a median of 12.112 seconds. The three paired whole-model savings were 12.172, 12.112, and 7.596 seconds. A simple resampling check placed the median saving between 7.596 and 12.172 seconds for these three pairs. Every optimized run was faster than its paired baseline.

Most importantly, all six substantive final CSV output files were **byte-for-byte identical** in every pair. That is stronger than saying the choices merely looked similar: the saved files matched character for character. The experiment also pins the exact ActivitySim source revision, configurations, patches, and hashes, so another person can repeat the same setup.

This is the honest current headline: on this workstation, for this public 50,000-household full-geography workload, ChoiceForge made the complete model 1.064 times faster while reproducing its reported final outputs exactly. It does not mean every ActivitySim model, computer, or future version will gain the same amount.

Why the next bottleneck is utility calculation

Destination choice works in two broad stages. First, the model evaluates many utility equations: for each potential destination it combines travel time, cost, household facts, destination facts, and rules. Second, it reduces those scores into probabilities and a selected destination.

ChoiceForge already makes the second stage faster. But the Phase 11 profiler showed that preparing and sending the first stage's finished utility values involved about 12,564,936 rows, or roughly 2.111 gigabytes of data. Sending a huge finished answer sheet to the GPU is like carrying every completed math problem to a very fast calculator. The more ambitious improvement is to send the compact ingredients and calculate the utility scores on the GPU too.

Phase 12: proving the ingredients can be translated safely

ActivitySim utility equations are written as expressions such as "travel time times a coefficient, plus an income term, only when a condition is true." A GPU cannot safely accelerate these expressions merely by guessing what they mean. The order of arithmetic, special functions, missing values, and table lookups all matter.

Phase 12 therefore built three safety layers:

1. An **expression reader** turns the public trip-mode-choice formulas into a small checked tree of operations. It successfully parses and evaluates all 253 distinct expressions used in that configuration on both CPU and GPU test paths.
2. A **skim adapter** gives the GPU path access to travel-time and cost lookup tables in the same way the model asks for them. Its tests confirm the adapter returns the expected values.
3. A **shadow capture** runs the GPU calculation beside the trusted CPU/Sharrow calculation without changing the official model answer. It records any difference for investigation.

The good news is that the deterministic GPU kernel exactly matched the local ordered CPU expression evaluator in the shadow test. The caution is that a compiled path can use different input precision, operation grouping, reduction order, or optional speed transformations such as `fastmath`. Any of these can change rounding in the last few binary digits. The shadows found differences from tiny fractions up to 0.25 for extremely large "unavailable" scores. No GPU utility result is allowed to replace Sharrow's production result until one explicit cross-device policy controls both targets.

A controlled speed test: encouraging, but not a production claim

Phase 12 also tested a deliberately simple 250,000-row, 64-feature, 21-alternative utility-and-choice pipeline. The CPU NumPy reference took 235.215 milliseconds. The GPU lowering plus 21-mode reduction took 28.988 milliseconds, an **8.114x** pipeline speedup, while the logsum check stayed within 0.00000000001.

This result answers a narrow question: can the compact-ingredient design be fast enough to matter? Yes. It does **not** yet prove an 8.114x whole-model improvement, because the test is not the full ActivitySim destination component and the CPU reference is not Sharrow's compiled production evaluator.

The strict IR plan: one recipe, two kitchens

The project is now pursuing a more rigorous solution rather than trying to tune away discrepancies one at a time. A **strict intermediate representation**, or strict IR, is a precise shared recipe for an equation. Instead of separately interpreting an expression for the CPU and generating a similar-looking CUDA kernel for the GPU, both targets are built from the same checked list of operations and numeric rules.

An everyday analogy: two kitchens can make the same named dish but use different measuring cups and a different order of steps. A strict recipe states the ingredients, measurements, order, and rounding rules so both kitchens produce the same dish. For ChoiceForge, the two kitchens are the strict CPU evaluator and the generated CUDA target.

The strict IR has generated a canonical description of the public MTC trip-mode utility: 379 terms across 21 alternatives. Phase 13 completed the strict CPU target, including separate ordered multiply and add steps, exact comparison reports, and fail-closed policy checks. Phase 14 completed the CUDA target generated from the same IR. The project test suite now passes 75 tests, including exact cross-device edge cases, compact skim gathering, and a device-resident handoff to the nested-logsum reducer.

This remains a project implementation, not a completed upstream Sharrow feature. Phase 15 removed the qualification-only utility transfer, connected generated utilities to the real logsum path, and ran direct A/B trials. It proved a

repeated destination-component gain at 1,001 households, but the 50,000-household candidate was slower than Phase 11. The strict path therefore remains opt-in instead of becoming the production default.

How to read the project status today

Question	Best current answer
Is there a real, repeated, exact speedup?	Yes. Phase 11 achieved 1.064x for the complete 50k-household, 1,454-zone public run, with byte-identical outputs in three pairs.
Is the destination component itself faster?	Yes. Its Phase 11 median was 1.388x faster.
Has the large utility equation been connected to the real GPU path?	Yes, as an opt-in Phase 15 candidate with Sharrow fallback. It is not the production default because the 50k gate is slower.
Is the compact GPU utility approach promising?	Yes. It was 8.114x faster in a controlled microbenchmark with a tight numerical gate.
What prevents a bigger claim today?	At 50k, the compact kernel performs too many scattered skim reads and uses one GPU block per row. It is exact but slower than Sharrow.

18. Phase 13: building the exact CPU answer key

Phase 12 discovered that saying "use the same formula" was not precise enough. Two compilers may change when a number is rounded or combine a multiplication and addition. The answers usually remain close, but a travel model can place a random draw very near a choice boundary. That makes the last few digits important.

Phase 13 turns the arithmetic rules into a written and executable contract. The strict CPU evaluator is like an official answer key. It requires:

- 64-bit arithmetic while an expression is being calculated;
- one conversion of the finished feature to a 32-bit number;
- 32-bit coefficients and utility totals;
- the original term order;
- a separate multiplication and addition for every term; and
- normal IEEE rounding, with speed shortcuts disabled.

If the IR version, policy, or identifying hash changes unexpectedly, the evaluator refuses to run. It also refuses unresolved coefficients and malformed arrays. These are useful failures: they stop a plausible-looking but undefined calculation from entering the model.

Did it cover the real model?

Yes. The canonical public MTC utility contains 379 terms for 21 travel-mode alternatives. Every term and alternative executes under the strict policy. An independent, simple scalar loop produced exactly the same utility bits. The complete repository now passes 75 tests.

Phase 13 then ran the public full-geography model with 1,001 households. It observed 30 real trip-mode batches containing 85,126 rows. That meant comparing 32,262,754 individual feature values and 1,787,646 utility values. ActivitySim completed all 34 model steps normally in 95.511 seconds, and Sharrow remained the official source of every model answer.

What did the comparison teach us?

The new strict answer key and current Sharrow did not match exactly. They agreed on 99.9118 percent of feature cells and 66.8085 percent of utility cells. The largest feature difference was about 0.0000305. The largest utility difference was 0.25, occurring among very large scores where a float32 number has relatively wide spacing.

Those percentages do not mean Sharrow is "66.8 percent correct." Sharrow follows its existing compiled arithmetic, while the strict evaluator follows the newly published cross-device policy. Many different utility cells can result from one tiny feature difference or a different order of hundreds of additions. Phase 13's job is to expose and classify that fact, not silently declare one existing compiler's behavior universal.

Every observed difference now has a stage. The reports found 28,466 feature cells whose compiled-expression result did not follow the strict expression policy. Among 593,346 different utility cells, 183 were explained by those input differences alone, while 593,163 also reflected a different accumulation rule. The first feature difference was about 0.00000763; the first utility difference was one float32 step.

Why Phase 13 is successful even though Sharrow differs

The goal was to create one stable answer key for future CPU and GPU code, not to copy whatever rounding happens on this workstation. All 379 terms and 21 alternatives run under that answer key. The comparison gate identifies the first different row, expression, alternative, values, and arithmetic stage. It can operate in observation mode, where Sharrow remains authoritative, or exact mode, where any difference stops qualification.

Phase 14 has now generated CUDA from this same strict IR and matched the strict CPU arrays exactly. Current Sharrow remains the safe ActivitySim fallback. The project will attempt another complete-model performance claim only after the generated path is integrated without qualification overhead and passes repeated byte-identical trials.

19. Phase 14: giving the GPU the exact same recipe

Phase 13 made the official answer key. Phase 14 built a code generator that turns that same checked recipe into a CUDA program. This matters because two separately handwritten programs can look equivalent while making tiny different choices about types, rounding, or operation order. Generating both evaluators from one hashed recipe greatly reduces that ambiguity.

What does the generated GPU program do?

For each model row, one GPU work group evaluates all 379 travel terms. It stores those temporary feature values in fast shared memory. Then separate GPU workers calculate the 21 travel-mode scores. Each worker uses the coefficients in the original order and performs a separate 32-bit multiplication and addition for every term. The resulting 21 scores can remain on the GPU and go directly into the nested-logsum calculation instead of taking a round trip through ordinary computer memory.

Inputs are packed by their real meaning: decimal numbers, whole numbers, and true/false values are not silently mixed. The compiled program is cached using the recipe hash, input types, coefficients, generated source, and compiler rules. Change any important ingredient and ChoiceForge builds a different kernel instead of reusing a stale one.

A tiny-number rule discovered by testing

Phase 14 found a useful edge case. A 32-bit floating-point number can be so tiny that it enters a special range called **subnormal**. The NVIDIA runtime on this machine turns such numbers into signed zero during these calculations. This is called **flush to zero**.

Ignoring the behavior would leave the phrase "exactly the same arithmetic" with a hidden exception. Instead, strict IR version 3 publishes the rule. The CPU answer key now also turns float32 subnormals into signed zero after the same steps. Tests deliberately use subnormal numbers, NaN ("not a number"), and infinity. This makes the cross-device agreement a real contract rather than an agreement that only holds for convenient inputs.

Did it match on the public model?

Yes. The same public 1,001-household, full-geography ActivitySim workload produced 30 trip-mode batches and 85,126 rows:

Exact Phase 14 check	Result
Batches	30 of 30
Feature values	32,262,754 of 32,262,754
Utility values	1,787,646 of 1,787,646
Largest feature difference	0.0
Largest utility difference	0.0
Terms and alternatives per batch	379 and 21

Every checked GPU bit matched the strict CPU answer key. ActivitySim still used Sharrow's established output as the official answer, so the experiment could not alter model behavior. That is the safe way to qualify a new backend.

Is it faster?

The generated kernel's diagnostic median was about 5.868 milliseconds. Preparing and transferring its inputs took about 116.157 milliseconds, and downloading the large comparison results took about 1.593 milliseconds. These numbers are not a fair speed contest. Phase 14 intentionally captures every one of the 379 feature columns so it can prove exactness term by term, while production code would keep compact inputs and useful outputs on the GPU.

Therefore the honest result is: **Phase 14 proves exact generated CPU/GPU semantics, not a new end-to-end speedup.** A separate test already confirms that the generated 21-column utility matrix can stay on the GPU through the nested-logsum reducer with no utility download and no reducer re-upload.

What does this change?

Before Phase 14, different arithmetic rules blocked the more ambitious GPU utility path. That obstacle is now resolved for this IR and hardware stack. That was the remaining Phase 15 problem: remove shadow-only transfers, connect generated utilities to the real path, and repeat public-model A/B runs. The next section explains what happened when those tests were completed.

20. Phase 15: putting the exact GPU recipe into the real model

Phase 14 proved that the CPU and GPU could follow the same recipe. Phase 15 asked a harder question: can the GPU's answer actually replace Sharrow's answer inside ActivitySim, stay on the graphics card for the next calculation, and make the real model faster?

The first connection worked, but exposed a data problem

The first candidate made all 379 ingredients as ordinary columns before the GPU kernel ran. Imagine copying every number from a library of road-time maps onto one enormous worksheet, sending the worksheet to a calculator, and then throwing it away. The arithmetic was fast, but preparing the worksheet was not.

At 1,001 households, the first repeated combined test looked faster than plain ActivitySim. A fairer test then compared Phase 15 directly with Phase 11, so both sides received the same destination batching and GPU nested-logsum work. That test showed the first strict utility path itself was slower. Measuring the parts found the cause: looking up and materializing skim columns took about 15.35 seconds across three candidate runs, while the useful GPU work took much less time.

The compact skim design

ChoiceForge was changed so the generated kernel receives:

- the shared road-time and cost map cubes;
- the already-checked origin, destination, and time positions for each row;
- the smaller ordinary chooser columns; and
- the strict coefficients and recipe.

The kernel now looks up a needed map value directly. It does not build a giant 379-column feature table. Identical map cubes are uploaded only once, even when ActivitySim creates new temporary wrapper objects. After trip destination is finished, the cache is released so later model steps do not pay for unused GPU memory. If any part fails, ChoiceForge uses Sharrow instead.

Did compact GPU gathering stay exact?

Yes. The final qualification covered 30 real public-model batches:

Check	Exact result
Rows	85,126
Feature values	32,262,754 of 32,262,754
Utility values	1,787,646 of 1,787,646
Largest strict CPU/GPU difference	0.0
Utility downloads before nested logsum	0 bytes
Nested-logsum utility re-uploads	0 bytes

Every modeled trip decision also matched. A printed diagnostic called `destination_logsum` differed by at most 0.000008 at this scale. That field is not used as a later decision in this output check. The difference comes from the published strict arithmetic versus current Sharrow arithmetic; the strict CPU and GPU answers still match bit for bit.

The direct repeated speed result

Three fresh pairs compared Phase 11 with compact Phase 15 at 1,001 households. The destination component improved from a median 11.3 seconds to 10.3 seconds, or **1.097 times faster**. Every Phase 15 destination run was faster than every Phase 11 destination run. All 90 strict utility batches stayed on the GPU for the nested calculation, and all modeled decisions matched.

The complete model did not pass the stricter promotion rule. Its median was 84.631 seconds for Phase 11 and 85.445 seconds for Phase 15. Other small model steps varied enough to hide the one-second destination gain. ChoiceForge therefore claims component superiority at this qualification scale, not a new whole-model record.

What happened at the large scale?

The final diagnostic used 50,000 households, all 1,454 zones, and 4,188,312 utility rows. Decisions still matched, but compact Phase 15 took 33.3 seconds for trip destination versus 28.4 seconds for Phase 11. The complete model took 197.871 versus 192.519 seconds. This is a clear rejection, so running three more expensive pairs would not create a responsible speed claim.

The reason is now narrower and useful. The GPU kernel gives one block to each row and makes many scattered reads from large skim cubes. At 50,000 households, that access pattern is slower than Sharrow's compiled expression path. The

next compiler should process tiles of nearby rows, combine repeated map reads, reuse a gathered value across multiple terms, and keep the exact ordered arithmetic rules.

What should a high-school reader conclude?

An experiment can succeed even when the final answer is "do not deploy this version." Phase 15 produced working, exact, fail-safe GPU integration and a repeated component win. It also used a public large benchmark to prove where the design stops winning. The production headline therefore remains Phase 11's repeated 50,000-household result. Phase 15 is the tested bridge to a better future compiler, not an excuse to replace a faster working system today.